MoveIt Random Movement

MoveIt Random Movement
- 1. Usage Environment
- 2. Driving the Real Machine
  - 3. Launch MoveIt2
    - 3.1 Random Movement
      - Startup Command
    - 3.2 Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine is achieved by subscribing to the Moveit2 `/joint_states` topic, converting the joint state information of the robotic arm into control data for the actual robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

The Jetson-Nano motherboard requires multi-machine communication configuration. The robotic arm and the virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in the Docker container. (Both are pre-configured with the same ros_domain_id at the factory; simply ensure the virtual machine and the robotic arm are on the same network.)

3. Launch MoveIt2

```
# Launch this command in the virtual machine,
ros2 launch dofbot_moveit demo.launch.py
```

```
yahboom@yahboom-VM:~$ ros2 launch dofbot_moveit demo.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-03-13-14-31-56-635718-yahboom-VM-9623
[INFO] [launch]: Default logging verbosity is set to INFO
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
[INFO] [robot_state_publisher-1]: process started with pid [9625]
[INFO] [move_group-2]: process started with pid [9627]
[INFO] [rviz2-3]: process started with pid [9629]
[INFO] [ros2_control_node-4]: process started with pid [9631]
[INFO] [spawner-5]: process started with pid [9633]
[INFO] [spawner-6]: process started with pid [9635]
[INFO] [spawner-7]: process started with pid [9637]
[robot_state_publisher-1] [WARN] [1773383518.815694424] [kdl_parser]: The root link base_link has an inertia specified in the URDF,
but KDL does not support a root link with an inertia.  As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-1] [INFO] [1773383518.815824354] [robot_state_publisher]: got segment Camera_Link
[robot_state_publisher-1] [INFO] [1773383518.815886691] [robot_state_publisher]: got segment Gripping_point_Link
[robot_state_publisher-1] [INFO] [1773383518.815895376] [robot_state_publisher]: got segment Llink1_Link
[robot_state_publisher-1] [INFO] [1773383518.815901628] [robot_state_publisher]: got segment Llink2_Link
[robot_state_publisher-1] [INFO] [1773383518.815907310] [robot_state_publisher]: got segment Llink3_Link
```

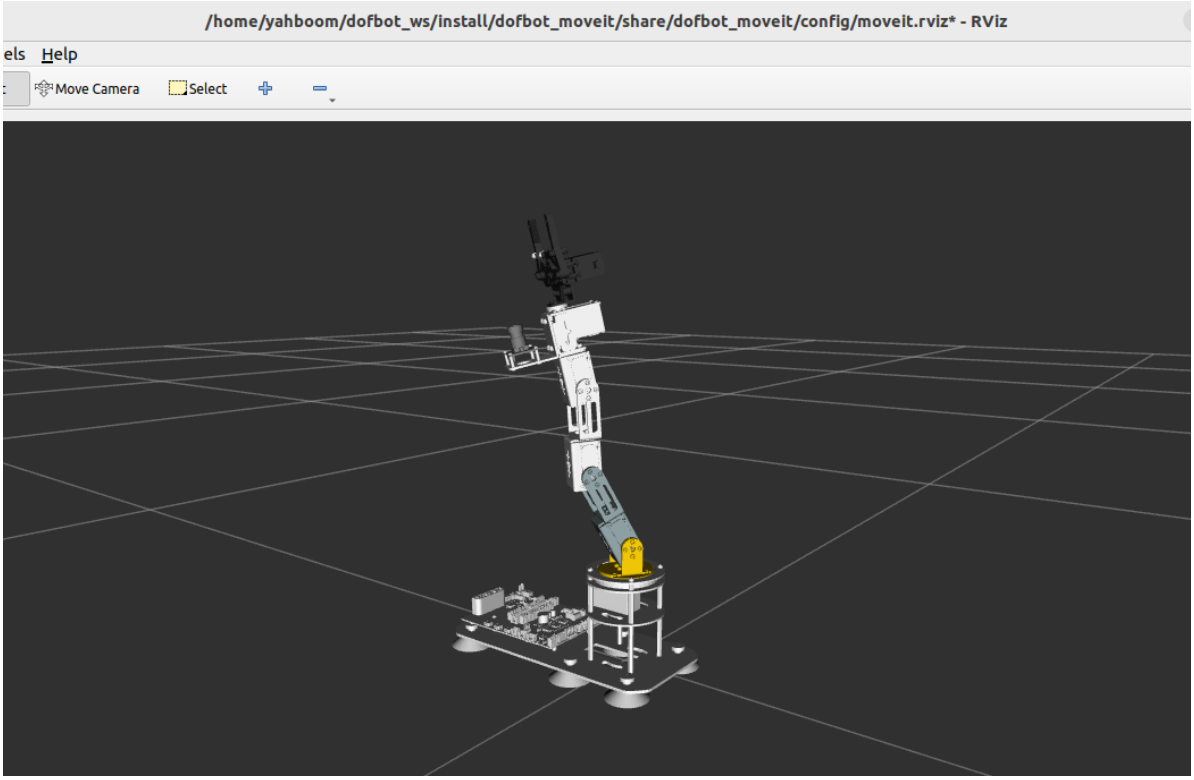
3.1 Random Movement

This random movement demo is not recommended for use on a real machine, as some poses may encounter obstacles or hit the motherboard! **

Startup Command

The robotic arm needs to be successfully loaded in MoveIt and display `You can start planning now!`. Run the following command: The robotic arm will automatically plan a random action.

```
#Start this command in the virtual machine,
ros2 run dofbot_moveit random_move
```



3.2 Source Code Analysis

Source code path:

```
#Path in the Virtual Machine
~/dofbot_ws/src/dofbot_moveit/src/random_move.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <geometry_msgs/msg/pose.hpp>
#include <random>

class RandomMove : public rclcpp::Node
{
public:
  RandomMove()
    : Node("random_target_move")
  {
    // Initialize other content
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {
    // Initialize move_group_interface_ and planning_scene_interface_ in this
function.
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    move_group_interface_->setNumPlanningAttempts(10);  // Set the maximum
number of planning attempts to 10
    move_group_interface_->setPlanningTime(5.0);         // Set the maximum
planning time per attempt to 5 seconds.

    // Plan Path
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;
    // Move the robotic arm to the preset UP pose.
    move_group_interface_->setNamedTarget("up");
    bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
    if (success)
    {
        RCLCPP_INFO(this->get_logger(), "Init arm successed.");
        move_group_interface_->execute(my_plan);
    }
    else
    {
        RCLCPP_ERROR(this->get_logger(), "Init arm failed!");
    }

    // Set random target position
    move_group_interface_->setRandomTarget();


    bool success_random = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);

    if (success_random)
    {
      RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
```

```
      move_group_interface_->execute(my_plan);
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(), "Planning failed!");
    }

  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<RandomMove>();

  // Lazy Initialization
  node->initialize();

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```